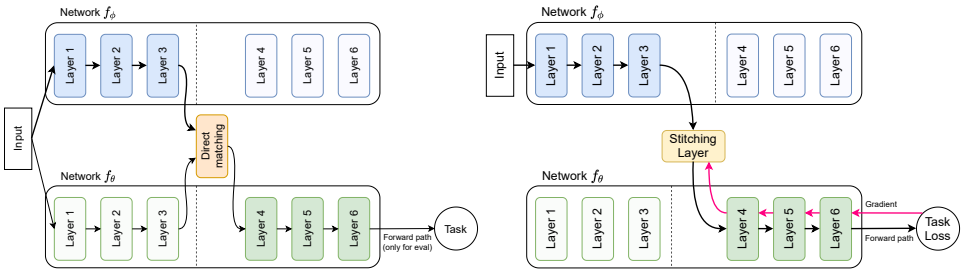


(a) Direct matching of representations. (b) Task loss matching of representations.

Figure 1: Illustrating two types of matching approaches. **(a) Direct matching**: first align the representations using only the representations themselves, and use the forward path on the task map of Model 2 — with the transformed activations — only to evaluate the performance of the stitched model. **(b) Task loss matching**: the forward path from the input starts on Model 1, then through the stitching layer it continues on Model 2, then task loss gradients are backpropagated to update the weights of the stitching layer.

Definition 3.2 (Frankenstein learning task)**.** *Given two pretrained neural networks f_θ and f_ϕ, a task given by a labeled dataset $D = \{(x_i, y_i) \in \mathcal{X} \times \mathcal{Y} : i = 1, \ldots, n\}$ and a loss function $\mathcal{L} : \mathcal{Y} \times \mathcal{Y} \to \mathbb{R}$, we will call the* Frankenstein learning task at layer L *the task of finding the stitching layer $S : \mathcal{A}_{\theta,L} \to \mathcal{A}_{\phi,L}$ which minimizes the total value of the loss $\mathcal{L}$ on the dataset D.*

Frankenstein is an evaluation framework The Frankenstein learning task primarily provides an *evaluation framework* for matching representations, and it does not tie our hands on how the transformation function S is produced. As an evaluation metric, we can either use the mean loss value of the Frankenstein learning task on an evaluation set, or any other appropriate metric (e.g., accuracy in the case of classification tasks).

Stitching convolutional representations Our toolset is generic, but here we will only consider convolutional architectures. Thus, the activation spaces have the structure of rank-3 tensors $\mathbb{R}^{w \times h \times c}$, where w and h are the spatial width and height and c is the number of feature maps. We will only consider linear stitching layers of the form $M : \mathbb{R}^c \to \mathbb{R}^c$ (1×1 convolutional layers). When formulating least squares problems on n many activation tensors, we always mean the least squares problem in $\mathbb{R}^c$, which in practice means reshaping the $n \times w \times h \times c$ tensor to a $nwh \times c$ matrix.

4 Methods to solve the Frankenstein learning task

Here we line up several possibilities to solve the Frankenstein learning task presented in the previous section. We distinguish two approaches: **(1) Direct matching**: utilize only the outputs of representation maps; **(2) Task loss matching**: utilize the loss of the Frankenstein learning task. Figure 1 illustrates the difference between the two approaches schematically.

4.1 Task loss matching

From the exposition of the Frankenstein learning task, a training method naturally follows: utilize the loss function in the Frankenstein learning task itself, and train an appropriately parametrised stitching layer with backpropagation (leaving the other parts of the Frankenstein network fixed).

We use the outputs of Model 2 as a soft label [Hinton et al., 2015] and define the task loss with cross-entropy. Another meaningful option would be to use the original training objective (the one that was used for the training of Model 1 and Model 2, e.g., the one-hot labels in a classification task). Both are reasonable choices as tasks, one corresponding to the stitched network imitating Model 2, the other to solving the task itself. We have experimented with both losses and found very high correlation, with no difference regarding our main observations. We often present performance in terms of relative accuracy compared to Model 2, because it is easier to interpret than cross-entropy.

4.2 Direct matching methods

Due to their close connection to similarity indices of representations, we also consider methods that arrive at the transformation S by using only the representations themselves, namely, utilizing only the outputs $A, B \in \mathbb{R}^{n \times p}$ of the representation maps of Model 1 and Model 2, respectively.

Least squares methods Given $A, B \in \mathbb{R}^{n \times p}$, and a class of transformations $\mathcal{C} \subseteq \mathrm{Hom}(\mathbb{R}^p, \mathbb{R}^p)$, we will consider least squares problems of the following kind: find an optimal $M_o \in \mathcal{C}$, such that

$$\|AM_o - B\|_F = \min_{M \in \mathcal{C}} \|AM - B\|_F. \tag{1}$$

We consider three variants of this problem: 1. arbitrary linear maps: $\mathcal{C} = \mathrm{Hom}(\mathbb{R}^p, \mathbb{R}^p)$, 2. orthogonal transformations: $\mathcal{C} = O(p)$, 3. linear maps in $\mathrm{Hom}(\mathbb{R}^p, \mathbb{R}^p)$ of rank at most k: $\mathcal{C} = \Sigma_k$.

In each case, there is a closed form expression relying on singular value decomposition. For $\mathcal{C} = \mathrm{Hom}(\mathbb{R}^p, \mathbb{R}^p)$, the minimal value of (1) is obtained through an orthogonal projection in the space of matrices [Penrose, 1956],

$$M_o = M_{LS} := A^{\dagger} B, \tag{2}$$

where $A^{\dagger}$ denotes the Moore-Penrose pseudoinverse of A. We will refer to this M_{LS} as the *least squares matching*. For results of this type, see Section 5.

In the case $\mathcal{C} = O(p)$, problem (1) is also known as the *orthogonal Procrustes problem*, see e.g. Golub and Van Loan [2013]. If $A^T B = USV^T$ is a singular value decomposition, then the optimal orthogonal map is $M_o = UV^T$.

Finally, the case $\mathcal{C} = \Sigma_k$ is also known as *reduced rank regression* Izenman [1975]. In terms of the SVD of $AM_{LS} = USV^T$, the optimal rank k map M_o is given by $M_o = M_{LS} V_k V_k^T$, where V_k consists of the first k columns of V. For reduced rank matching results, see Section 7.

Sparse matchings It is known [Wang et al., 2018] that representations are not fully local, that is, matching cannot be achieved by permutation matrices. High quality sparse matchings between neurons can be interpreted to imply "slightly distributed" representations [Li et al., 2016].

To achieve sparsity, a natural approach is to add the regularization term $\|M\|_1 = \sum |m_{ij}|$ to the loss function, where M is the transformation matrix. Then one can consider the L1 regularized versions of 1) task loss matching, and 2) least squares matching (Lasso regression, as in Li et al. [2016]):

$$\mathcal{L}(A, B) = \|AM - B\|_F + \alpha \cdot \|M\|_1, \tag{3}$$

where A, B denote the activations of the two models. See Section 8 for these experiments.

5 Matching representations in deep convolutional networks

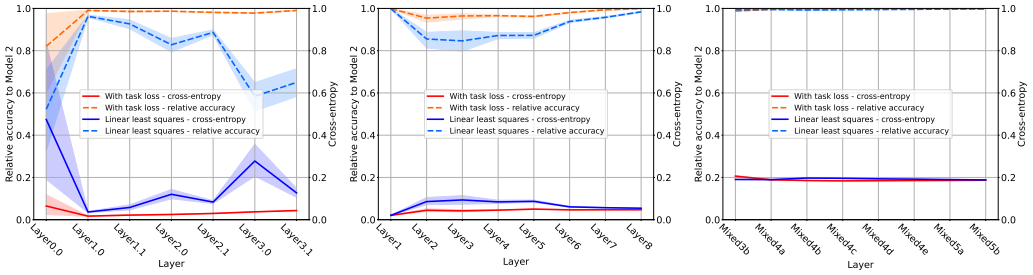
We now continue our exposition by presenting a series of experiments regarding matching neural representations. Our results in this section can be summarized in the following statement:

Neural representations arising on a given layer of convolutional networks that share the same architecture but differ in initialization can be matched with a single affine stitching layer, achieving close to original performance on the stitched network.

5.1 Experiment 1: Matching with least squares and task loss matching

In this experiment, we compare the performance of direct matching and task loss matching. We take two networks of identical architectures, but trained from different initializations. We perform a direct matching as well as a task loss matching from Model 1 to Model 2. We then compare the performance of the stitched network to the performance of Model 2.

We conduct experiments on three different convolutional architectures: a simple, 10-layer convnet called Tiny-10 (used in Kornblith et al. [2019]), on a ResNet-20 [He et al., 2016], and on the Inception V1 network [Szegedy et al., 2015]. These networks are rather standard and represent different types of architectures. Tiny-10 and ResNet-20 are trained and evaluated on CIFAR-10, Inception V1 on the 40-label CelebA task. Further training details follow common practices (see Appendix A.3).



(a) Results with ResNet-20. (b) Results with Tiny10. (c) Results with Inception V1.

Figure 2: Performance of Frankenstein networks stitched together at specific layers (horizontal axis) measured in relative accuracy and cross-entropy (left and right vertical axes, respectively). This experiment compares two methods: task loss matching and linear least squares matching (a direct matching method). Results are averages of 10 runs, the bands represent standard deviations. As baseline values for cross-entropy, we provide average cross-entropies between Model 2 and a model stitched without transformation: Tiny-10: 6.99 ± 1.96, ResNet-20: 4.69 ± 2.91, InceptionV1: 0.36 ± 0.08 (averaged across all layers).

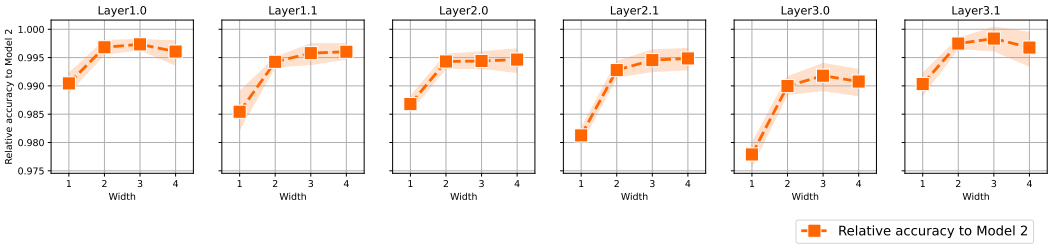


Figure 3: Relative accuracy values with ResNet-20 nets of different width. The horizontal axis shows the width multiplier compared to the baseline network. Plots correspond to different layers with names denoted at the top. Averages of 10 runs, bands show standard deviations.

Figure 2 summarizes the results: task loss matching achieves near perfect accuracy on ResNet-20, and also succeeds in aligning representations on Tiny-10 without collapsing accuracy. While the direct matching significantly underperforms the task loss matching and collapses performance on several layers, it is still remarkable that direct matching performs fairly well in many cases: e.g., on the layers between Layer1.0 and Layer2.1 on ResNet-20 or almost every case on Tiny-10.

Further results We have experimented with networks with batch normalization and without batch normalization, and networks trained with different optimizers (SGD, Adam). Still, we did not encounter any cases where there were unmatchable instances. The worst performing stitchings were after the first convolutional layer on ResNet-20, still achieving 80% relative accuracy. While this method is not universal on arbitrary architectures, this does not modify the observation that 'matchability' of representations is a quite robust property of common vision models.

In the case of task loss matching, we found the most stable results when we initialized the training of the stitching layer from the least squares matching (2). See Section 8 on further details on how the initialization influences performance. We also evaluated Procrustes matching described in Section 4.2, and found that on later layers, it achieves the same level of accuracy as linear least squares matching.

As detailed in Section C.2 of the Appendix, accuracy drop during stitching can be surprisingly low even when we stitch networks trained on different datasets; in our case, ImageNet and CelebA.

5.2 Experiment 2: Networks of different width

This experiment focuses on the effect of network width on stitching performance. Matching happens with cross-entropy to Model 2 output as the task loss. The stitching is initialized to the optimal least squares matching. Performance is measured on the original classification task, relative to Model 2 performance. Figure 3 shows the results on ResNet-20 with various widths. The relative accuracy for 1x-width is already very high, but a clear trend of performance increasing with width is still visible.